

Software Requirements Specifications for *Rogue-Like Algebra*

Version 2.0

November 24, 2025

Prepared by:

Fall CS411W Team Gold

Consisting of:

Brennen Gabriel, Gabriel Lugo-Maldonado,

Jillian Cequeria, Brandon Thomas,

Jacob Thompson, and Bryan Baker

Old Dominion University

Table of Contents

Table of Contents	2
1. Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 Definitions, Acronyms, and Abbreviations	3
1.4 References	4
1.5 Overview	5
2 Overall Description	6
2.1 Product Perspective	6
2.2 Product Functions	6
2.3 User Characteristics	6
2.4 Constraints	6
2.5 Assumptions and Dependencies	6
3 System Requirements	7
3.1 Rogue-Like Algebra Features	7
3.1.1 Rogue-Like Algebra: Level Design (O: Thompson)	7
3.1.2 Rogue-Like Algebra: Enemy Characters	8
3.1.3 Rogue-Like Algebra: Problem Generation	8
3.1.4 Rogue-Like Algebra: Tile maps	9
3.2 Performance Requirements	10
3.3 Design Constraints	10
3.3.1 Compatibility Requirement (O: Thompson)	10
3.3.2 Scalability Requirement (O: Thompson)	10
3.4 Software System Attributes	10
3.5 Other Requirements	10
3.5.1 Copyright Compliance (O: Thompson)	10
3.6.1 Random Question Generation System (O: Brennen)	10

1. Introduction

Section 1 and the subsections therein provide the necessary contexts to understand the requirements of *Rogue-like Algebra* (e.g. definitions and references).

1.1 Purpose

This Software Requirements Specifications document will provide detailed descriptions for specifications and requirements for *Rogue-like Algebra*, a video game designed to aid in the learning of various algebraic concepts.

1.2 Scope

In our current world, students are finding it harder to learn and understand algebraic concepts through the traditional ways of teaching algebra. To try to solve this issue, a solution should address the issues that make the traditional means of teaching ineffective by being an interactive and more engaging medium of learning such concepts. *Rogue-like Algebra* aims to teach algebraic concepts that students find themselves struggle to absorb through engaging and fun gameplay in the rogue-like genre.

1.3 Definitions, Acronyms, and Abbreviations

- Adaptive Learning – An educational method that adjusts content difficulty based on the learner’s performance to personalize the learning experience.
- Algebraic Concepts – Foundational mathematical principles involving variables, equations, expressions, and functions typically taught in middle and high school.
- Continuous Integration (CI) – A development practice where developers frequently merge code changes into a shared repository, triggering automated builds and tests to detect issues early.
- Continuous Deployment (CD) – A software release process that automatically delivers validated code changes to production, minimizing manual intervention and ensuring faster updates.
- GDScript – A Python-like scripting language used in the Godot game engine, designed for creating game logic.
- Git – A distributed version control system that tracks changes in source code during software development, enabling collaboration and history tracking.
- GitHub – A cloud-based platform for hosting Git repositories. It offers tools for collaboration, issue tracking, code review, and CI/CD integration.
- Godot – A free and open-source game engine used for developing 2D and 3D games, known for its flexibility and ease of use.

- Integrated Development Environment (IDE) – A software suite that provides tools like a code editor, debugger, and compiler to streamline software development (e.g., VS Code).
- Procedural Generation – A method of creating data algorithmically rather than manually, often used for generating levels, puzzles, or content dynamically in games.
- Rogue-like – A subgenre of games typically featuring procedural generation, turn-based movement, and permanent death, emphasizing strategy and replayability.
- Role-playing Game (RPG) – A genre of game in which a character assumes a role to complete various goals whether it be a larger story-line, or less connected quests, or both.
- Rust – A systems programming language known for performance, safety, and concurrency.
- Turn-Based Combat – A gameplay mechanic where players and opponents take alternating turns to perform actions such as attacking or defending.
- Visual Studio Code (VS Code) – A popular, lightweight code editor developed by Microsoft, used for writing and debugging code.

1.4 References

- “About Us - DragonBox / WeWantToKnow.” Dragonbox.com, dragonbox.com/about.
- Prodigy. “Prodigy Math | Prodigy Education.” Wwww.prodigygame.com, www.prodigygame.com/main-en/prodigy-math/.
- “IXL - Membership.” Wwww.ixl.com, www.ixl.com/membership.
- Steam. “Steam Database.” SteamDB, steamdb.info/.
- “Top Roguelike Games by Sales and Revenue on Steam as of 2022 – Steam Marketing Tool.” Games-Stats.com, games-stats.com/steam/?tag=roguelike.
- Decker-Woodrow, Lauren E, et al. The Impacts of Three Educational Technologies on Algebraic Understanding in the Context of COVID-19. Vol. 9, 1 Jan. 2023, p. 233285842311659-233285842311659, <https://doi.org/10.1177/23328584231165919>.
- Aine, Sandip, and Maxim Likhachev. “Anytime Truncated D* : Anytime Replanning with Truncation.” Proceedings of the International Symposium on Combinatorial Search, vol. 4, no. 1, 20 Aug. 2021, pp. 2–10, <https://doi.org/10.1609/socs.v4i1.18295>
- Guy, Stephen, and Ioannis Karamouzas. Guide to Anticipatory Collision Avoidance.
- “NavigationMesh.” Godot Engine Documentation, 2025, docs.godotengine.org/en/stable/classes/class_navigationmesh.html

“Creating Your First 2D Game with A* Algorithm.” HackerEarth Blog, 6 Oct. 2016, hackerearth.com/blog/developers/creating-first-2d-game-algorithm/

Team Gold. “Lab 1 – Rogue-like Algebra.” Team Gold, 6 Oct. 2025, <https://gold-team-odu.github.io/labs>

1.5 Overview

Section 2 of this document will provide an overview of *Rogue-like Algebra*.

Section 3 of this document will provide a list of requirements that the product should fulfill.

2 Overall Description

Section 2 and the subsections contained within discuss *Rogue-like Algebra* in more detail with the overall perspective, functions, and those who will be using it.

2.1 Product Perspective

Rogue-like Algebra is a web-based application that is designed to be widely accessible on any web browser to help in the idea of being an effective way for people to learn and help build core algebraic principles. Featuring elements from various genres of video games including RPGs and rogue-likes, *Rogue-like Algebra* aims to add a twist of algebra to the gameplay loop many are familiar with to help those who struggle with learning such concepts.

2.2 Product Functions

Rogue-like Algebra will be a web-based application that will feature concepts that are common to the top-down RPG genre of video games including a character the player controls in different states of the game (e.g. overworld game state or battle game state) with enemy characters that behave according to the state of the game and reactive to the player character's actions. The various enemy characters will be able to engage the player in turn-style battle where procedural problems are created for the player to end the battle victoriously. The player will be able to encounter various equipment and power-ups in their gameplay whether it be from an active battle or the more passive exploration of the world.

2.3 User Characteristics

Rogue-like Algebra has three major groups of users who benefit the most from the educational goals the game aims to assist in: Students (or anybody seeking to improve their algebra skills, but students will be the biggest part of this group), those who can assist in first group's teachings, being teachers and parents, and the wider video game consuming audience. Students can use this software with the intent to either learn or strengthen knowledge of algebraic principles using the medium's higher engagement capabilities.

2.4 Constraints

N/A

2.5 Assumptions and Dependencies

N/A

3 System Requirements

Section 3 and the subsections contained within outline the various requirements that *Rogue-Like Algebra* must abide by.

3.1 *Rogue-Like Algebra* Features

The requirements and specifications listed in this subsection relate to the various workings of how the system shall handle the various gameplay related features.

3.1.1 *Rogue-Like Algebra*: Level Design (O: Thompson)

3.1.1.1 The system shall provide the capability to generate a new dungeon layout at the start of each game session, ensuring that:

- Each room connects to at least one other room.
- At least one valid path exists from the dungeon's entry to its exit.
- Layout, room size, and contents offered vary based on a random seed value.
- A predefined seed value may be used to reproduce a previous layout.

3.1.1.2 The system shall generate a random seed value to either create a new dungeon layout or retrieve a previous one.

3.1.1.3 The system shall adapt the layout of the dungeon proportionately to the difficulty of the problems.

- As the difficulty increases, the aesthetic of the dungeon becomes more treacherous and dangerous.

3.1.1.4 The system shall occasionally give the player a choice between two paths.

- The choices between paths will be randomly generated based on the random seed value.
- The frequency of a choice being offered will be randomly generated based on the random seed value.
- The choices that can be made include choosing different types of questions, opting to take a risk for better reward, or choosing a safe option (healing the player).

3.1.1.5 The system shall adapt the aesthetic of the dungeon based on the player's performance and choice.

- The better the player is doing, the more difficult and treacherous the dungeon will appear, darkening the colors, using more gothic textures, and making the BGM more grim.

3.1.1.6 The system shall prevent the player from moving out of the boundaries of the dungeon.

3.1.1.7 The system shall spawn appropriate enemies based on the difficulty and dungeon aesthetic that the player is currently in.

- The better the player is doing, greater, stronger enemies will spawn. The worse the player does, lesser, weaker enemies will spawn.

3.1.2 *Rogue-Like Algebra: Enemy Characters*

3.1.2.1 The system shall include enemy characters placed throughout the overworld map of the game to act as an interactible challenge. (O: Bryan)

- Enemies can be avoidable, but some important enemies will not be to act as a benchmark for the strength of the player.
 - These important enemies will be in places such as at the end of a room or dungeon.

3.1.2.2 The system shall provide the capability for the game to switch game states if the player character has collided with an enemy character to the battle state. (O: Bryan)

3.1.2.4 The system shall implement various states for enemy characters to utilize to create dynamic movement including but not limited to: (O: Bryan)

- Idle state in which an enemy is not aware of the player's existence
- Chase state in which the player has entered an enemy's detection radius
- Defeated state in which the player has exited the battle state successfully defeating the enemy

3.1.2.5 The system shall provide rewards for defeating enemies based on their difficulty, selective importance to a room's context (e.g. defeating a specific enemy to open a door to another room), or thematic importance. (O: Bryan)

- Difficulty will be measured through an enemy's fortitude in a battle scene through the difficulty in problems posed and number of problems posed.
- Rewards include in-game resources such as currencies or in-game items.

3.1.3 *Rogue-Like Algebra: Problem Generation*

3.1.3.1 The system shall generate algebraic problems to display to the user, with the first problem generated containing only one variable and one integer. (O: Jillian, M1: Bryan)

3.1.3.2 The system shall solve the current problem and hold onto the correct answer for comparison. (O: Jillian, M1: Bryan)

3.1.3.3 The system shall allow user input for answering each problem with a text input box. (O: Jillian, M1: Bryan)

3.1.3.4 The system shall take the user input and compare it against the correct answer to the current problem. (O: Jillian, M1: Bryan)

3.1.3.5 The system shall output the user input, the correct answer, and whether the user was correct or incorrect in a formatted display. (O: Jillian, M1: Bryan)

3.1.3.6 The system shall provide the capability to increase the difficulty of the next problem if the user input is correct by doing one of the following: (O: Jillian, M1: Bryan)

- Inserting a new random integer into the problem (e.g. + 5, - 12)
- Inserting a new random variable into the problem (e.g. x, y)

3.1.3.7 The system shall provide the capability to decrease the difficulty of the next problem if the user input is incorrect by doing one of the following: (O: Jillian, M1: Bryan)

- Removing an integer from the problem if the number of current integers = more than one
- Removing a variable from the problem if the number of current variables = more than one

Constraints

- Each generated problem must be validated to ensure it has exactly one correct solution.
- Each problem must be generated and verified within 150 milliseconds to maintain smooth gameplay.

3.1.4 *Rogue-Like Algebra: Tile maps*

3.1.4.1 The system shall provide the capability for the designer to place tiles on a grid map to create a visual representation of the world. (O: Bryan, M1: Lugo-Maldonado)

Constraints

- Tiles must include walls and floors.
- Interactables must include chests and doors.
- Tiles must be dictated by tile map sprites.
- Wall tiles must not be traversable by the player.
- Empty tiles must not be traversable by the player.
- Tiles should include variant sprites.

3.1.5 *Rogue-Like Algebra: Character Controller* (O: Lugo-Maldonado)

The Character Controller is implemented to provide the Player the capability to control their Character through actions. This must include movement and world interaction.

Constraints

- The control inputs must support both keyboard & mouse and standard gamepads as input options.
- Interaction with the world should happen using a single shared input action rather than several different input actions for each different type of interaction. (For example, using the “E” key to interact with a chest, door, etc. Not “C” for chest, “D” for door, etc.)
- The Player should be able to remap input actions to different buttons.
- The Player must be able to move around using the WASD keys/Left analog stick.
- The Player must be able to interact with the world using the “F” key/A button on standard gamepads.
- The Player must not be able to move through solid objects in the world, such as walls.
- The camera must stay locked to the Player.

3.2 Performance Requirements

3.3 Design Constraints

3.3.1 Compatibility Requirement (O: Thompson)

3.3.1.1 The software must be compatible with Windows and Linux, and all major web browsers such as Google Chrome, Mozilla Firefox, Safari, and Microsoft Edge.

3.3.2 Scalability Requirement (O: Thompson)

3.3.2.1 The system architecture must be designed to accommodate an increasing user base and database traffic without loss of functionality.

3.4 Software System Attributes

3.5 Other Requirements

3.5.1 Copyright Compliance (O: Thompson)

3.5.1.1 Any content used by the development team such as images, audio and text must properly credit the source to abide by copyright laws.

3.6.1 Random Question Generation System (O: Brennen)

3.6.1.1 The system automatically generates algebra problems to ensure each play session provides a unique experience. This feature maintains player engagement while reinforcing different algebraic concepts. The requirements are listed:

3.6.1.1.1 The system shall dynamically generate algebra problems by selecting random numerical values and operators.

3.6.1.1.2 The system shall randomly choose question types from predefined categories including, but not limited to, linear equations, quadratic equations, and inequalities.

3.6.1.1.3 The system shall verify that each generated problem has exactly one correct solution.

3.6.1.1.5 The system shall prevent the repetition of identical problems within a single play session.

3.6.1.2 Constraints

- **Unique Solution Constraint:** Each generated problem must be validated to ensure it has exactly one correct solution.
- **Performance Constraint:** Each problem must be generated and verified within 150 milliseconds to maintain smooth gameplay